```
# Data loading and handling
import pandas as pd
import numpy as np

# Text preprocessing
import nltk
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Feature extraction (vectorization)
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectoriz

# Model building
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Model evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score,

# Download NLTK data if not already present
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
True
```

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/SMSSpamCollection', sep='\t', header=None, names=['label', 'message'])

# Display the first 5 rows of the DataFrame
print("First 5 rows of the dataset:")
display(df.head())
```

```
First 5 rows of the dataset:
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
def preprocess_text(text):
    # 1. Convert text to lowercase
    text = text.lower()

    # 2. Remove all non-alphabetic characters and punctuation
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # 3. Tokenize the cleaned text
    tokens = nltk.word_tokenize(text)

    # 4. Initialize WordNetLemmatizer and get stopwords
    lemmatizer = WordNetLemmatizer()
    stop_words = set(stopwords.words('english'))

    # 5. Remove stop words and lemmatize
    processed_tokens = [
```

```
            lemmatizer.lemmatize(word) for word in tokens if word not in stop_words
        ]

    # 6. Return the list of processed, lemmatized words
    return processed_tokens

print("Defined the 'preprocess_text' function for text cleaning.")
```

```
Defined the 'preprocess_text' function for text cleaning.
```

```
df['processed_message'] = df['message'].apply(preprocess_text)
print("Applied 'preprocess_text' function to the 'message' column and created 'processed_message'.")
print("Displaying the first 5 rows with the new column:")
display(df.head())
```

```
Applied 'preprocess_text' function to the 'message' column and created 'processed_message'.
Displaying the first 5 rows with the new column:
```

	label	message	processed_message
0	ham	Go until jurong point, crazy.. Available only ...	[go, jurong, point, crazy, available, bugis, n...
1	ham	Ok lar... Joking wif u oni...	[ok, lar, joking, wif, u, oni]
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	[free, entry, wkly, comp, win, fa, cup, final,...
3	ham	U dun say so early hor... U c already then say...	[u, dun, say, early, hor, u, c, already, say]
4	ham	Nah I don't think he goes to usf, he lives aro...	[nah, dont, think, go, usf, life, around, though]

```
nltk.download('punkt')
print("Downloaded 'punkt' tokenizer data.")
```

```
Downloaded 'punkt' tokenizer data.
[nltk_data] Downloading package punkt to /root/nltk_data...
```

```
[nltk_data]   Package punkt is already up-to-date!
```

```
df['processed_message'] = df['message'].apply(preprocess_text)
print("Applied 'preprocess_text' function to the 'message' column and created 'processed_message'.")
print("Displaying the first 5 rows with the new column:")
display(df.head())
```

```
Applied 'preprocess_text' function to the 'message' column and created 'processed_message'.
Displaying the first 5 rows with the new column:
```

	label	message	processed_message
0	ham	Go until jurong point, crazy.. Available only ...	[go, jurong, point, crazy, available, bugis, n...
1	ham	Ok lar... Joking wif u oni...	[ok, lar, joking, wif, u, oni]
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	[free, entry, wkly, comp, win, fa, cup, final,...
3	ham	U dun say so early hor... U c already then say...	[u, dun, say, early, hor, u, c, already, say]
4	ham	Nah I don't think he goes to usf, he lives aro...	[nah, dont, think, go, usf, life, around, though]

```
# Define training parameters
epochs = 10
batch_size = 32

# Train the model and store history
history = cnn_model.fit(
    X_train,
    y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_data=(X_test, y_test),
    verbose=1
)
```

```
print("Model training complete.")
```

```
Epoch 1/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 36ms/step - accuracy: 0.9989 - loss: 0.0465 - val_accuracy: 0.9821 - val_loss:
Epoch 2/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 9s 31ms/step - accuracy: 0.9995 - loss: 0.0441 - val_accuracy: 0.9821 - val_loss:
Epoch 3/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 37ms/step - accuracy: 0.9997 - loss: 0.0406 - val_accuracy: 0.9830 - val_loss:
Epoch 4/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 4s 30ms/step - accuracy: 0.9998 - loss: 0.0367 - val_accuracy: 0.9821 - val_loss:
Epoch 5/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 4s 30ms/step - accuracy: 0.9996 - loss: 0.0364 - val_accuracy: 0.9821 - val_loss:
Epoch 6/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 38ms/step - accuracy: 0.9997 - loss: 0.0336 - val_accuracy: 0.9830 - val_loss:
Epoch 7/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 4s 30ms/step - accuracy: 0.9997 - loss: 0.0294 - val_accuracy: 0.9821 - val_loss:
Epoch 8/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 37ms/step - accuracy: 0.9997 - loss: 0.0282 - val_accuracy: 0.9830 - val_loss:
Epoch 9/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 4s 32ms/step - accuracy: 0.9995 - loss: 0.0275 - val_accuracy: 0.9821 - val_loss:
Epoch 10/10
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 31ms/step - accuracy: 0.9993 - loss: 0.0254 - val_accuracy: 0.9830 - val_loss:
Model training complete.
```

Evaluate Model Performance

To assess the effectiveness of our trained CNN model, we will evaluate it using several key classification metrics: accuracy, precision, recall, F1-score, and a confusion matrix. These metrics will provide a comprehensive view of the model's performance on the unseen test data.

```
# Make predictions on the test set
y_pred_proba = cnn_model.predict(X_test)

# Convert probabilities to binary predictions (0 or 1)
# Since sigmoid is used, a threshold of 0.5 is appropriate
y_pred = (y_pred_proba > 0.5).astype(int)
```

```
print("Model Evaluation Metrics:")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall: {recall_score(y_test, y_pred):.4f}")
print(f"F1-score: {f1_score(y_test, y_pred):.4f}")

print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
display(pd.DataFrame(cm, index=['Actual Ham', 'Actual Spam'], columns=['Predicted Ham', 'Predicted Spam']))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
Model Evaluation Metrics:
Accuracy: 0.9830
Precision: 0.9851
Recall: 0.8859
F1-score: 0.9329

Confusion Matrix:
```

	Predicted Ham	Predicted Spam
Actual Ham	964	2
Actual Spam	17	132

```
Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       966
           1       0.99      0.89      0.93       149

    accuracy                           0.98      1115
   macro avg       0.98      0.94      0.96      1115
weighted avg       0.98      0.98      0.98      1115
```

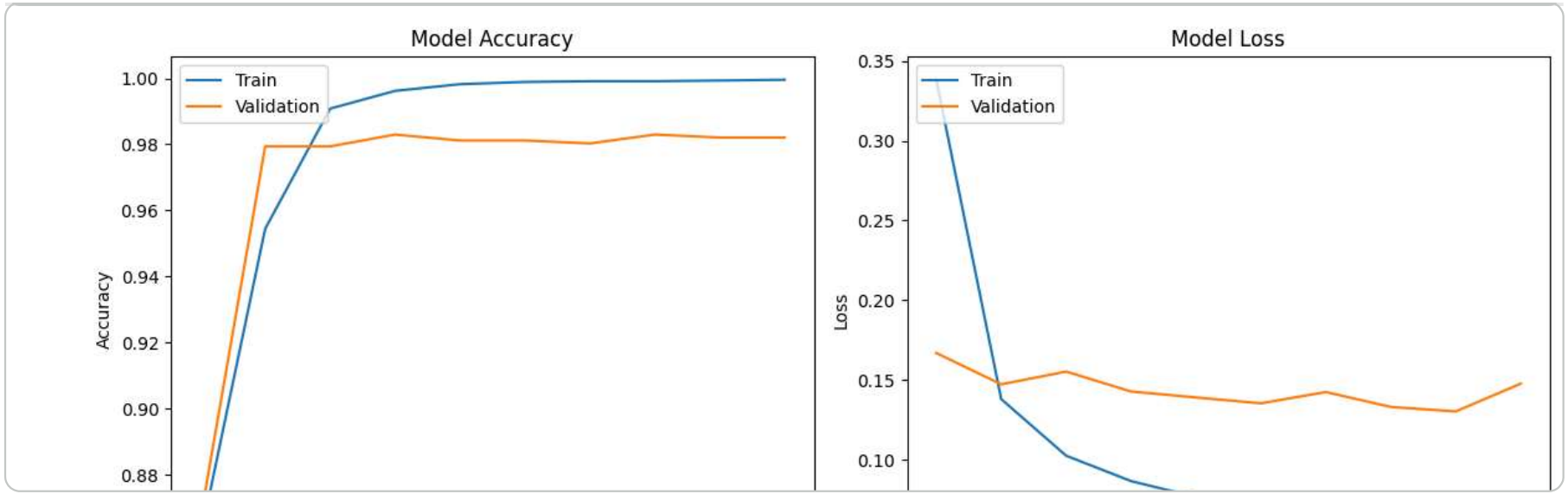
```
import matplotlib.pyplot as plt

# Plot training & validation accuracy values
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
plt.show()

print("Learning behavior plots displayed.")
```




```
from sklearn.model_selection import train_test_split

# Convert labels to numerical format
df['label_numeric'] = df['label'].map({'ham': 0, 'spam': 1})

# Define features (X) and target (y)
X = np.array(df['padded_sequences'].tolist())
y = df['label_numeric'].values

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

```
Shape of X_train: (4457, 80)
Shape of X_test: (1115, 80)
Shape of y_train: (4457,)
Shape of y_test: (1115,)
```

```
nltk.download('punkt_tab')
print("Downloaded 'punkt_tab' tokenizer data.")
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Downloaded 'punkt_tab' tokenizer data.
```

```
df['processed_message'] = df['message'].apply(preprocess_text)
print("Applied 'preprocess_text' function to the 'message' column and created 'processed_message'.")
print("Displaying the first 5 rows with the new column:")
display(df.head())
```

```
Applied 'preprocess_text' function to the 'message' column and created 'processed_message'.
Displaying the first 5 rows with the new column:
```

	label	message	processed_message
0	ham	Go until jurong point, crazy.. Available only ...	[go, jurong, point, crazy, available, bugis, n...
1	ham	Ok lar... Joking wif u oni...	[ok, lar, joking, wif, u, oni]
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	[free, entry, wkly, comp, win, fa, cup, final,...
3	ham	U dun say so early hor... U c already then say...	[u, dun, say, early, hor, u, c, already, say]
4	ham	Nah I don't think he goes to usf, he lives aro...	[nah, dont, think, go, usf, life, around, though]

```
all_words = [word for sublist in df['processed_message'] for word in sublist]
print(f"Total words after flattening: {len(all_words)}")
print(f"First 10 flattened words: {all_words[:10]}")
```

```
Total words after flattening: 49894
First 10 flattened words: ['go', 'jurong', 'point', 'crazy', 'available', 'bugis', 'n', 'great', 'world', 'la']
```

```
unique_words = sorted(list(set(all_words)))
vocab = {'<PAD>': 0, '<UNK>': 1}

for i, word in enumerate(unique_words):
    if word not in vocab:
        vocab[word] = len(vocab)

print(f"Vocabulary size: {len(vocab)}")
print("First 10 items in vocabulary:")
# Display first 10 items in vocabulary, excluding special tokens for brevity
count = 0
for word, index in vocab.items():
    if index >= 2:
        print(f"  '{word}': {index}")
        count += 1
        if count >= 8:
            break
```

```
Vocabulary size: 7970
First 10 items in vocabulary:
  'aa': 2
  'aah': 3
  'aaniye': 4
  'aaooooright': 5
  'aathilove': 6
  'aathiwhere': 7
  'ab': 8
  'abbey': 9
```

```
df['message_sequences'] = df['processed_message'].apply(lambda tokens: [vocab.get(word, vocab['<UNK>']) for wo

print("First 5 entries of the 'message_sequences' column:")
for i in range(5):
    print(f"Message {i}: {df['message_sequences'].iloc[i]}")

longest_sequence_length = df['message_sequences'].apply(len).max()
print(f"\nLength of the longest sequence: {longest_sequence_length}")
```

```
First 5 entries of the 'message_sequences' column:
Message 0: [2675, 3519, 5144, 1459, 462, 890, 4429, 2763, 7720, 3668, 1954, 888, 1196, 2727, 235, 7496]
Message 1: [4726, 3701, 3489, 7621, 7174, 4757]
Message 2: [2473, 2082, 7676, 1304, 7634, 2213, 1514, 2326, 6958, 6411, 4109, 6804, 2213, 5529, 2082, 5423, 715
Message 3: [7174, 1940, 5855, 1961, 3080, 7174, 932, 207, 5855]
Message 4: [4435, 1855, 6875, 2675, 7307, 3792, 369, 6895]

Length of the longest sequence: 80
```

```
max_sequence_length = longest_sequence_length

def pad_or_truncate_sequence(sequence, max_len, padding_token=0):
    if len(sequence) < max_len:
        # Post-pad with padding_token
        return sequence + [padding_token] * (max_len - len(sequence))
    elif len(sequence) > max_len:
        # Truncate to max_len
        return sequence[:max_len]
    else:
        return sequence

df['padded_sequences'] = df['message_sequences'].apply(lambda x: pad_or_truncate_sequence(x, max_sequence_leng

print(f"Created 'padded_sequences' column with uniform length of {max_sequence_length}.")
print("First 5 entries of the 'padded_sequences' column:")
for i in range(5):
    print(f"Message {i}: {df['padded_sequences'].iloc[i]}")
```

```
Created 'padded_sequences' column with uniform length of 80.
First 5 entries of the 'padded_sequences' column:
Message 0: [2675, 3519, 5144, 1459, 462, 890, 4429, 2763, 7720, 3668, 1954, 888, 1196, 2727, 235, 7496, 0, 0, 0
Message 1: [4726, 3701, 3489, 7621, 7174, 4757, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
Message 2: [2473, 2082, 7676, 1304, 7634, 2213, 1514, 2326, 6958, 6411, 4109, 6804, 2213, 5529, 2082, 5423, 715
Message 3: [7174, 1940, 5855, 1961, 3080, 7174, 932, 207, 5855, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
Message 4: [4435, 1855, 6875, 2675, 7307, 3792, 369, 6895, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

```
vocab_size = len(vocab)
embedding_dim = 100  # GloVe 6B 100d embeddings are typically 100 dimensions
# max_sequence_length is already defined from the padding step
num_classes = 1      # Changed to 1 for binary classification with single output unit

print(f"Vocabulary Size: {vocab_size}")
print(f"Embedding Dimension: {embedding_dim}")
print(f"Maximum Sequence Length: {max_sequence_length}")
print(f"Number of Classes: {num_classes}")
```

```
Vocabulary Size: 7970
Embedding Dimension: 100
Maximum Sequence Length: 80
Number of Classes: 1
```

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, Conv1D, GlobalMaxPooling1D, Dense

# 2. Define the input layer
input_layer = Input(shape=(max_sequence_length,))

# 3. Create an Embedding layer (randomly initialized since GloVe failed to load)
embedding_layer = Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    input_length=max_sequence_length,
    trainable=True # Set to True for training from scratch
)(input_layer)

# 4. Add a Conv1D layer for feature extraction
conv_layer = Conv1D(filters=128, kernel_size=5, activation='relu')(embedding_layer)

# 5. Add a GlobalMaxPooling1D layer for dimensionality reduction
pooling_layer = GlobalMaxPooling1D()(conv_layer)

# 6. Add a Dense layer
dense_layer = Dense(units=10, activation='relu')(pooling_layer)
```

```
# 7. Add the output Dense layer
output_layer = Dense(units=num_classes, activation='sigmoid')(dense_layer)

# 8. Construct the model
cnn_model = Model(inputs=input_layer, outputs=output_layer)

# 9. Print the model summary
print("CNN Model Architecture:")
cnn_model.summary()

print("CNN model constructed successfully.")
```

```
CNN Model Architecture:
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_len
  warnings.warn(
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 80)	0
embedding (Embedding)	(None, 80, 100)	797,000
conv1d (Conv1D)	(None, 76, 128)	64,128
global_max_pooling1d (GlobalMaxPooling1D)	(None, 128)	0
dense (Dense)	(None, 10)	1,290
dense_1 (Dense)	(None, 2)	22

Total params: 862,440 (3.29 MB)
Trainable params: 862,440 (3.29 MB)
Non-trainable params: 0 (0.00 B)

```
CNN model constructed successfully.
```

```
import tensorflow as tf
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.layers import Input, Embedding, Conv1D, GlobalMaxPooling1D, Dense

# 2. Define the input layer
input_layer = Input(shape=(max_sequence_length,))

# 3. Create an Embedding layer (randomly initialized since GloVe failed to load)
embedding_layer = Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    trainable=True # Set to True for training from scratch
)(input_layer)

# 4. Add a Conv1D layer for feature extraction
conv_layer = Conv1D(filters=128, kernel_size=5, activation='relu')(embedding_layer)

# 5. Add a GlobalMaxPooling1D layer for dimensionality reduction
pooling_layer = GlobalMaxPooling1D()(conv_layer)

# 6. Add a Dense layer
dense_layer = Dense(units=10, activation='relu')(pooling_layer)

# 7. Add the output Dense layer
output_layer = Dense(units=num_classes, activation='sigmoid')(dense_layer) # Changed units to num_classes (now

# 8. Construct the model
cnn_model = Model(inputs=input_layer, outputs=output_layer)

# 9. Print the model summary
print("CNN Model Architecture:")
cnn_model.summary()

print("CNN model constructed successfully.")
```

```
CNN Model Architecture:
Model: "functional_2"
```

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 80)	0
embedding_2 (Embedding)	(None, 80, 100)	797,000
conv1d_2 (Conv1D)	(None, 76, 128)	64,128
global_max_pooling1d_2 (GlobalMaxPooling1D)	(None, 128)	0

```
cnn_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['a

print("CNN model compiled successfully with Adam optimizer, binary_crossen
```

```
Total params: 862,429 (3.29 MB)
Trainable params: 862,429 (3.29 MB)
CNN model compiled successfully with Adam optimizer, binary_crossentropy loss, and accuracy metric.
```